PRN:

Batch:

Experiment no 4

Title: Matrix Computations using OpenMP: Parallel Decomposition Strategies

1. **Objective:**
   To implement matrix computations using OpenMP and investigate how different problem decomposition strategies affect parallel algorithm design and performance.

2. **Use:**

One of the most important decisions in parallel programming is how to divide a problem into independent tasks. The same problem can often be decomposed in multiple ways, each leading to different levels of parallelism, communication overhead, and scalability.

This assignment introduces decomposition strategies that are widely used in HPC applications.

- **HPC in Practice**

Different decomposition strategies are employed in real-world applications:

Decomposition Strategy	Typical Applications
Input Data Decomposition	Image processing, matrix operations, deep learning training
Output Data Decomposition	Weather forecasting, finite element analysis, fluid dynamics
Recursive Decomposition	Merge sort, quicksort, FFT, Strassen matrix multiplication
Speculative Decomposition	Branch prediction, transaction processing, game-tree search, AI planning

Before writing any code:

1. Explain the difference between **task decomposition** and **data decomposition**.
2. Why is matrix multiplication considered highly parallelizable?
3. Which decomposition strategy would you choose for:
   - Image filtering
   - Merge sort
   - Chess engine
   - Weather simulation, Justify your answers.

Section A:

Task 1: Matrix-Vector Multiplication using Input Data Decomposition

Implement sequential and OpenMP versions of matrix-vector multiplication.

Decomposition Strategy

Each thread processes a subset of matrix rows.

Illustrate the decomposition with a diagram in your report.

Compare:

- Sequential execution time
- Parallel execution time
- Speedup
- Efficiency

Task 2: Matrix-Matrix Multiplication using Output Data Decomposition

Implement sequential and OpenMP versions.

Decomposition Strategy

Assign different blocks of the output matrix $C$to different threads.

Experiment with:

- Static scheduling
- Dynamic scheduling

Record performance for different matrix sizes.

Answer the following:

- Why is output decomposition preferable for matrix multiplication?
- What synchronization is required?
- How does workload balance affect performance?

Section B:

Task 3: Compare Decomposition Strategies

Using matrix multiplication, compare:

- Row-wise decomposition
- Column-wise decomposition
- Block decomposition (2D)

Evaluate:

- Load balancing
- Cache locality
- Execution time
- Scalability

Prepare a comparison table:

Strategy	Advantages	Limitations	Suitable Applications
Row-wise			
Column-wise			
Block			

Task 4: Performance Analysis

Execute the best-performing implementation using:

- 1 thread
- 2 threads
- 4 threads
- 8 threads
- Maximum available threads

Measure:

- Execution time
- Speedup
- Efficiency

Plot:

- Threads vs Execution Time
- Threads vs Speedup

Section C

Complete **any ONE**.

Challenge 1 – Recursive Decomposition

Choose one algorithm:

- Parallel Merge Sort
- Parallel Quick Sort
- Recursive Matrix Multiplication

Explain:

- How recursive decomposition is applied.
- How tasks are created.
- How recursion enables parallel execution.

Challenge 2 – Speculative Decomposition

Research one application that uses speculative decomposition.

Examples:

- Chess engines
- AI search algorithm
- Database transaction processing
- Branch prediction in processors

Prepare a one-page report explaining:

- Why speculation is needed.
- Benefits.
- Challenges (rollback, wasted computation, synchronization).

Challenge 3 – OpenMP Tasks

Reimplement one recursive algorithm using OpenMP task constructs and compare it with a sequential version.

Discuss:

- Task creation overhead
- Granularity
- Performance

Section D: AI / HPC Case Study

Choose one domain:

- Deep Learning
- Scientific Simulation
- Computer Vision
- Robotics
- Big Data Analytics

Identify:

1. Which decomposition strategy is used?
2. Why is it suitable?
3. How does it improve scalability?
4. Would you implement it using OpenMP, MPI, CUDA, or a hybrid approach?

Write a **300–500 word report**.

Answer the following questions:

1. Which decomposition strategy gave the best performance? Why?
2. How does decomposition influence scalability?
3. Which decomposition would you use for distributed-memory systems?
4. Which strategy is most suitable for GPUs?
5. Can one problem be solved using multiple decomposition strategies? Explain with an example.

Submit in a pdf:

- Source code for all implementations
- Execution outputs
- Performance analysis tables
- Performance graphs
- AI/HPC case study report
- Answers to the questions (total 12 questions)

